Enterprise Training on Modern C++, Linux Debugging, WASM & Rust Migration Training Program

Table of Contents

Modern Systems Programming: C++ → Rust → WASM 4

Day 1 — Advanced C++ Debugging on Linux (RHEL 9) 4

Session 1: Debugging Fundamentals (Advanced View) 4

Session 2: GDB Mastery 4

Session 3: Crash & Core Dump Analysis 4

Session 4: Memory Debugging 5

Session 5: Performance Debugging 5

Hands-on Lab 5

Day 2 — Modern C++ (C++20 / C++23) 5

Session 1: Modern Language Design 5

Session 2: C++20 Features 5

Session 3: Concurrency & Coroutines 5

Session 4: Modern Libraries 5

Session 5: C++23 Highlights 6

Hands-on Lab 6

Day 3 — Rust Programming Fundamentals 6

Session 1: Rust Philosophy 6

Session 2: Ownership Model 6

Session 3: Type System 6

Session 4: Error Handling 6

Session 5: Concurrency in Rust **6**

Hands-on Lab ..... **Error! Bookmark not defined.**

Day 4 — C++ to Rust Migration + Interoperability **7**

Session 1: Migration Strategy **7**

Session 2: C++ ↔ Rust Interop (FFI) **7**

Session 3: Unsafe Rust **7**

Session 4: Performance Comparison **7**

Session 5: Real Migration Case Study **7**

Hands-on Lab **7**

Day 5 — Web Assembly (WASM) + Integrated Architecture **8**

Session 1: WASM Fundamentals **8**

Session 2: Toolchain **8**

Session 3: WASI & Runtime **8**

Session 4: Performance & Security **8**

Session 5: End-to-End System Design **8**

Final Hands-on Project **8**

Deliverables **9**

Outcomes **9**

Key Value for Client **9**

Modern Systems Programming: C++ → Rust → WASM

Total Duration: 40 Hours(5days)

Target Audience

- Senior C/C++ developers
- Systems / backend engineers
- Embedded / infra engineers

Prerequisites

- Strong C/C++ fundamentals
- Linux (RHEL/Ubuntu) familiarity
- Basic understanding of build systems (Make/CMake)

Day 1 — Advanced C++ Debugging on Linux (RHEL 9)

Session 1: Debugging Fundamentals (Advanced View)

- Debugging optimized vs debug builds
- Symbol tables, DWARF info
- Stack vs heap corruption patterns

Session 2: GDB Mastery

- Breakpoints (conditional, hardware)
- Watchpoints and memory tracking
- Reverse debugging
- Debugging multi-threaded applications

Session 3: Crash & Core Dump Analysis

- Enabling core dumps (ulimit, systemd)
- Post-mortem debugging
- Backtrace analysis in real systems
- Hang dump analysis

Session 4: Memory Debugging

- Valgrind (memcheck, massif)
- AddressSanitizer (ASAN)
- UndefinedBehaviorSanitizer (UBSAN)
- Handle leak analysis type of handles

Session 5: Performance Debugging

- Perf (CPU profiling)
- Flame graphs
- Cache misses, branch prediction
- How to debug 100% cpu usage

Hands-on Lab

- Debug a crashing multithreaded service
- Identify memory leak using ASAN
- Analyze a real core dump

Day 2 — Modern C++ (C++20 / C++23)

Session 1: Modern Language Design

- RAII revisited
- Smart pointers deep dive
- Move semantics & ownership

Session 2: C++20 Features

- Concepts (type safety)
- Ranges (functional pipelines)
- Constexpr improvements

Session 3: Concurrency & Coroutines

- std::thread, async, futures
- Coroutines for async programming
- Structured concurrency

Session 4: Modern Libraries

- STL improvements
- fmt (formatting)

- spdlog (logging)
- Boost (selected modules)
- googletest (test framework)

Session 5: C++23 Highlights

- std::expected
- Improved ranges
- Deducing this

Hands-on Lab

- Refactor legacy C++ code to modern C++
- Implement async workflow using coroutines
- Replace raw pointers with smart pointers

Day 3 — Rust Programming Fundamentals

Session 1: Rust Philosophy

- Memory safety without GC
- Zero-cost abstractions

Session 2: Ownership Model

- Ownership, borrowing, lifetimes
- Move semantics vs C++

Session 3: Type System

- Structs, enums
- Pattern matching
- Traits (interfaces)

Session 4: Error Handling

- Result vs Option
- Panic vs recoverable errors

Session 5: Concurrency in Rust

- Fearless concurrency
- Send / Sync
- Mutex, channels

Hands-on Lab

- Rewrite a C++ module in Rust
- Fix memory bug using ownership model
- Implement thread-safe service

Day 4 — C++ to Rust Migration + Interoperability

Session 1: Migration Strategy

- When NOT to migrate
- Incremental vs full rewrite
- Risk management
- Migration techniques from legacy C++ to C++23

Session 2: C++ ↔ Rust Interop (FFI)

- Extern "C
- Binding generation (bindgen, cbindgen
- Calling Rust from C++

Session 3: Unsafe Rust

- When and why to use unsafe
- Memory layout compatibility

Session 4: Performance Comparison

- Rust vs C++ benchmarks
- Memory footprint analysis

Session 5: Real Migration Case Study

- Legacy module → Rust service
- Error reduction patterns

Hands-on Lab

- Wrap C++ library in Rust
- Build hybrid C++ + Rust system
- Replace unsafe C++ code with safe Rust

Day 5 — Web Assembly (WASM) + Integrated Architecture

Session 1: WASM Fundamentals

- WASM architecture
- Execution model
- Sandbox environment

Session 2: Toolchain

- LLVM → WASM
- Emscripten (C++)
- Rust → WASM (wasm-pack)

Session 3: WASI & Runtime

- WASI interface
- Running WASM outside browser
- Edge computing use cases

Session 4: Performance & Security

- Startup latency
- Sandboxing benefits
- Use cases in production

Session 5: End-to-End System Design

- C++ → Rust → WASM pipeline
- Multi-language architecture
- Microservices + WASM modules

Final Hands-on Project

Build a system:

- C++ module (legacy)
- Rust module (safe logic)
- WASM module (portable execution)
- Integrate all components into one system

Deliverables

Participants receive:

- Source code for all labs
- Debugging cheat sheets
- C++ → Rust migration guide
- WASM deployment templates
- Performance tuning checklist

Outcomes

By the end of training, engineers will:

- Debug production C++ systems efficiently
- Write modern C++ (C++20/23)
- Build safe systems using Rust
- Migrate C++ modules to Rust
- Deploy portable compute via WASM

Key Value for Client

This program enables transition from:

Legacy C++ systems

to

Modern, safe, portable architectures

using:

Modern C++ + Rust + WebAssembly